

 subject.pdf (<https://cdn.intra.42.fr/pdf/pdf/193132/en.subject.pdf>)

Preliminary tests

If cheating is suspected, the evaluation stops here. Use the "Cheat" flag to report it. Take this decision wisely, and please, use this button with caution.

Prerequisites

The code must compile with c++ and the flags -Wall -Wextra -Werror. Don't forget this project has to follow the C++98 standard. Thus, C++11 (and later) functions or containers are NOT expected.

Any of these means you must not grade the exercise in question:

- A function is implemented in a header file (except for template functions).
- A Makefile compiles without the required flags and/or another compiler than c++.

Any of these means that you must flag the project with "Forbidden Function":

- Use of a "C" function (*alloc, *printf, free).
- Use of a function not allowed in the exercise guidelines.
- Use of "using namespace <ns_name>" or the "friend" keyword.
- Use of an external library, or features from versions other than C++98.

 Yes

 No

Exercise 00: My First Class in Orthodox Canonical Form

This exercise introduces the notion of canonical class with a simple arithmetic example: the fixed-point class.

Makefile

There is a Makefile that compiles using the appropriate flags.

If this is not the case, the evaluation of this exercise stops here. you can move on to the next exercise.

 Yes

 No

Accessors

The Fixed class (or whatever its name) must provide accessors to the raw value:

- int getRawBits(void) const;
- void setRawBits(int const raw); Are these member functions present and functional?

 Yes

 No

Canonical

A canonical class must provide at least:

- A default constructor
- A destructor
- A copy constructor
- An copy assignment operator Are these elements present and functional?

 Yes

 No

Exercise 01: Towards a more useful fixed-point number class

The previous exercise was a good start, but the class was still pretty useless since it was only able to represent a fixed-point value 0.0.

Makefile

There is a Makefile that compiles using the appropriate flags.
If this is not the case, the evaluation of this exercise stops here. you can move on to the next exerc

☒ Yes ☐ No

Floating-point constructor

Is it possible to construct an instance from a floating-point value?

☒ Yes ☐ No

<< operator

Is there a << operator overload and is it functional?

☒ Yes ☐ No

Fixed-point value to integer value

A member function "int toInt(void) const;" that converts the fixed-point value to an integer value must be present. Is it functional?

☒ Yes ☐ No

Fixed-point value to floating point value

A member function "float toFloat(void) const;" that converts the fixed-point value to a float value must be present. Is it functional?

☒ Yes ☐ No

Integer constructor

Is it possible to construct an instance from an integer value?

☒ Yes ☐ No

Exercise 02: Now we are talking

This exercise adds comparison and arithmetic features to the class.

Makefile

There is a Makefile that compiles using the appropriate flags.
If this is not the case, the evaluation of this exercise stops here. you can move on to the next exerc

☒ Yes ☐ No

Comparison operators

Are the 6 comparison operators (>, <, >=, <=, == and !=) implemented and working properly?

☒ Yes ☐ No

Arithmetic operators

Are the 4 arithmetic operators (+, -, * and /) implemented and working properly?
(if you ever do a division by 0, it is acceptable that the program crashes)

☒ Yes ☐ No

Other operators

Are the pre-increment, post-increment, pre-decrement and post-decrement operators implemented and working properly?

✔ Yes

✖ No

Static member functions overloads

Last but not least, test the the min() and max() static member functions are implemented and working properly.

✔ Yes

✖ No

Exercise 03: BSP

This exercise should have make you realize how easy it is to implement complex algorithms once ti as intended.

Makefile

There is a Makefile that compiles using the appropriate flags.
If this is not the case, the evaluation of this exercise stops here. you can move on to the next exerc

✔ Yes

✖ No

Class Point

There is a class Point which has two attributes (x and y) of type Fixed const. It also has a constructor that takes two floats and initialize x and y with those values.

✔ Yes

✖ No

Function bsp

There is a function bsp() which prototype is "bool bsp(Point const a, Point const b, Point const c, Point const point)".
The function returns True if the point is inside the triangle described by the vertices a, b, and c.
It returns False otherwise.

✔ Yes

✖ No

Main and tests

There is at least a main to test that the function bsp() works as required.
Run several test to make sure that the return value is correct.

✔ Yes

✖ No

Ratings

Don't forget to check the flag corresponding to the defense

✔ Ok

★ Outstanding project

Empty work

📄 Incomplete work

⚙ Invalid compilation

📋 Cheat

⚠ Concerning situation

💧 Leaks

🚫 Forbidden function

💬 Can't support /

Conclusion

Leave a comment on this evaluation (2048 chars max)

Finish evaluation